

ADVANCED CONTEXT ENGINEERING

GraphRAG vs Vectorless *vs Vector RAG.*

A 2026 guide to advanced context engineering — three retrieval philosophies that share a name. 📌



Rohini Manoharan

Follow me for more dev & AI

+ Follow

THE CRACKS

Why vector RAG is breaking

It became the standard because it works — for simple lookup. Then questions get harder, and three failures appear it cannot escape:

 **No relationships** — can't follow “Section 4 cross-references Appendix C” — they live far apart in embedding space

 **Chunking destroys structure** — a 512-token window severs tables from headers, footnotes from numbers

 **Collapses with complexity** — past ~5 entities per query, accuracy degraded to 0% without a graph



Rohini Manoharan

Follow me for more dev & AI

+ Follow

THE BASELINE

How vector RAG works

The pipeline everyone builds first:

vector-rag

Document → Chunk (512) → Embed → Vector DB

Query → Embed → Similarity → Top-K → LLM → Answer

→ “Close enough” in embedding space $\neq$ “relevant.” Chunks lose neighbors; no way to traverse relationships.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

ARCHITECTURE 1

GraphRAG: when relationships ARE the answer

It doesn't replace vector search — it adds a layer vector search can't replicate: a map of how things connect. Entities become nodes, relationships become edges.

It captures “GDPR Article 17 is enforced by the European Data Protection Board” in a way similarity never could.

💡 The biggest myth: you need a pre-existing knowledge graph. You don't — the LLM builds it from your documents.



Rohini Manoharan

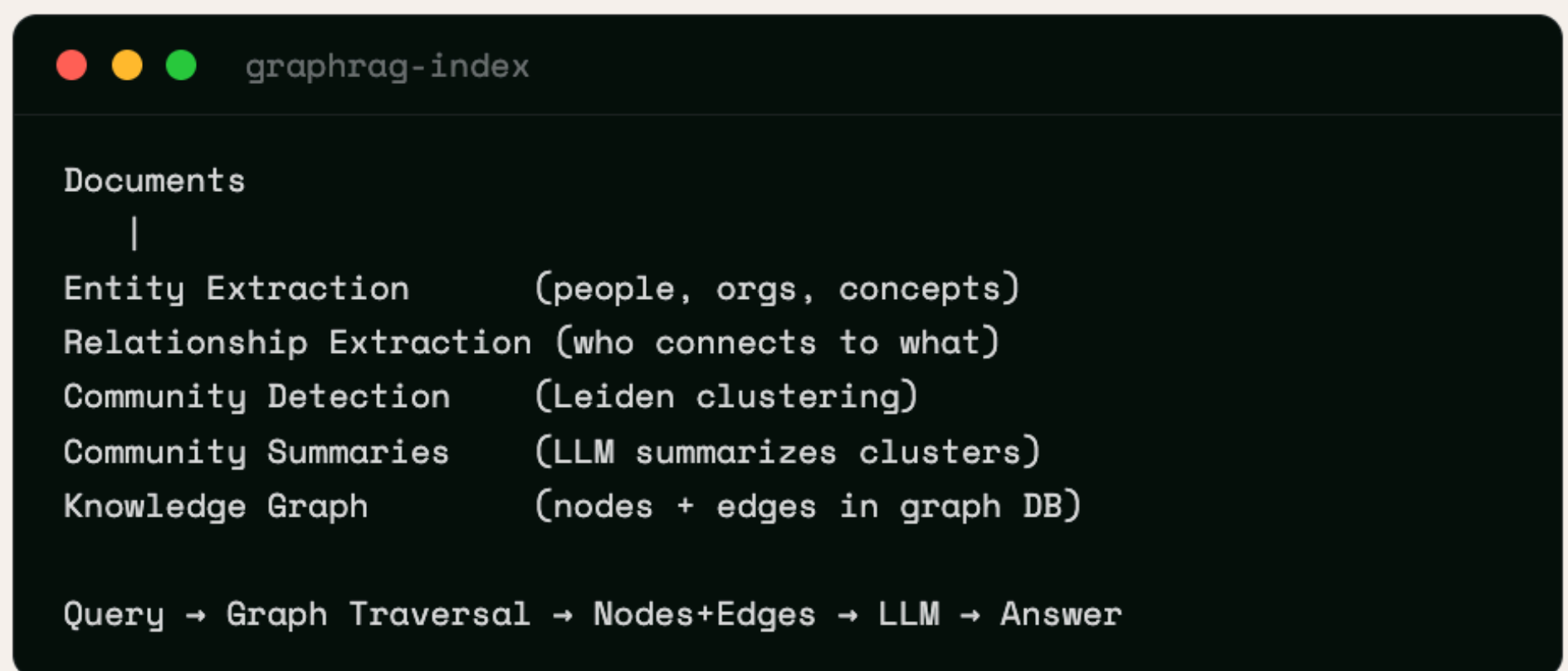
Follow me for more dev & AI

+ Follow

UNDER THE HOOD

How GraphRAG works

The indexing pipeline turns documents into a graph:



→ 72–83% comprehensiveness and a 3.4× accuracy improvement in enterprise scenarios.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

BE HONEST

GraphRAG: the tradeoffs

✅ **Shines at** — multi-hop questions, global synthesis across 500 papers, regulatory cross-references

💰 **Real cost** — \$20–500 to index a corpus; LazyGraphRAG cut it to 0.1% by deferring summaries (+2–8s/query)

❌ **Fails at** — simple lookup, fast-changing real-time data, small collections where infra outweighs benefit



Rohini Manoharan

Follow me for more dev & AI

+ Follow

ARCHITECTURE 2

Vectorless RAG: structure beats similarity

It took the harder path — what if the whole retrieval model is the wrong starting point? PageIndex (VectifyAI, 23K+ stars) borrows from AlphaGo: don't search exhaustively — navigate intelligently.

The LLM reasons about where the answer would live in the document's structure, then goes there — the way an expert scans a table of contents.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

UNDER THE HOOD

How Vectorless RAG works

No chunking. No embeddings. No vector DB. Just the document's natural hierarchy.

pageindex-tree

PDF → Tree Index (Chapter → Section → cells)

Chapter 1: Revenue

1.1 Q1 Results

Table: Revenue by Region

Footnotes: Currency adjustments

1.2 Q2 Results

Chapter 2: Expenses

Query → LLM inspects the ToC tree

→ "Revenue is in Chapter 1.1"

→ navigates there, cites exactly

→ It iterates if the answer is incomplete — reasoning its way through the tree.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

THE PROOF

Hard to argue with the numbers

On FinanceBench:

 Mafin 2.5 (PageIndex) — 98.7%

 Traditional vector RAG — ~50%

 Perplexity — ~45%

 GPT-4o, no RAG — ~31%

A 49-point gap. The catch: it swaps vector approximation for LLM-reasoning approximation — worth it only when accuracy is the constraint.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

THE NUANCE

They're NOT competing

The edge case most articles miss — they solve different problems:

 **GraphRAG** — understand relationships across a large collection

 **Vectorless** — precise answers from the internal structure of complex docs

 **Vector** — fast, cheap semantic lookup over unstructured text

None is a drop-in replacement. Each solves something the others genuinely can't.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

DECIDE FAST

The decision matrix

Match the architecture to the question:

decision-matrix		
Query type	Accuracy	Choose

Semantic lookup	Normal	Vector RAG
Multi-hop relations	High	GraphRAG
Structured doc exact	Very high	Vectorless
Global themes	Normal	GraphRAG
Cross-ref navigation	Very high	Vectorless
Mixed complexity	Varies	Adaptive RAG

→ Ask: what kind of questions? how much does accuracy matter?
what's the structure? latency/cost limits?



Rohini Manoharan

Follow me for more dev & AI

+ Follow

WHAT WINNERS SHIP

Adaptive RAG

No single architecture dominates. The winners aren't picking one — they're routing.

adaptive-rag

User Query → Complexity Classifier

Simple?	→ Vector RAG	(fast, cheap)
Relationship?	→ GraphRAG	(accurate)
Structured doc?	→ Vectorless	(precise)

→ More complex to build — but more accurate AND cheaper at scale, since most queries are simple.



Rohini Manoharan

Follow me for more dev & AI

+ Follow

SIT WITH THIS

Three retrieval philosophies

The teams that shipped broken RAG weren't bad engineers — they used vector RAG correctly. It just wasn't the right tool for the question.

 **Vector RAG** — optimistic matching — find what's close, hope it's right

 **GraphRAG** — structural mapping — know the relationships before the question arrives

 **Vectorless RAG** — deliberate navigation — reason about where the answer lives



Rohini Manoharan

Follow me for more dev & AI

[+ Follow](#)

The shift.

Not “does RAG work?” — “which architecture?”

In 18 months, “**vector or adaptive RAG?**” will separate production teams from demo teams — the way “do you have evals?” already does.

Which are you running — Vector, Graph, Vectorless, or Adaptive?



Repost

Save

Comment

#RAG · #GraphRAG · #ContextEngineering · #AIEngineering



Rohini Manoharan

Follow me for more dev & AI

+ Follow